

Introduction to Spatial Statistics

NSF REU Bootcamp - Last Day!

Monnie McGee, PhD

Southern Methodist University

June 10, 2026

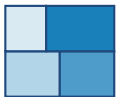
*“Near things are more related
than distant things.”*

— Waldo Tobler, 1970

What this means for statistics

- Most stats methods assume observations are **independent**
- Spatial data breaks that assumption — nearby locations tend to look similar
- We call this **spatial autocorrelation**
- Ignoring it leads to misleading p -values and conclusions

Three Kinds of Spatial Data



Areal

Values attached to
polygons

*Counties, zip codes, census
tracts*



Point process

Locations of events

*Disease cases, earthquakes,
crimes*



Geostatistical

Continuous surface,
sampled at points

*Soil chemistry, air quality
monitors*

We will work through a real example of each type today.

Part 1: Areal Data — NC SIDS

Dataset: North Carolina SIDS

- Sudden Infant Death Syndrome deaths
- 100 NC counties, 1974–1984
- Classic teaching dataset in spatial stats
- Built into the [spData](#) package

Our question:

Are high SIDS rates *clustered* in certain parts of the state, or are they scattered randomly?

Compute a rate

```
nc <- nc |>
  mutate(
    rate =
      SID74/BIR74*1000)
```

Deaths per 1,000 live births

Making a Choropleth Map

```
ggplot(nc) +  
  geom_sf(  
    aes(fill = sids_rate74),  
    color = "white") +  
  scale_fill_viridis_c(  
    option = "magma") +  
  theme_void()
```

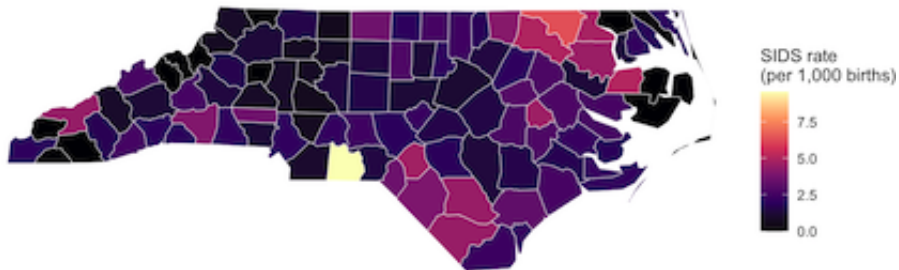
Choropleth: shade each polygon by its value. Darker = higher rate.

Discuss

Look at the map. Do high-rate counties seem to cluster together, or are they scattered randomly across the state?

North Carolina Choropleth Map

NC SIDS Rate, 1974–78

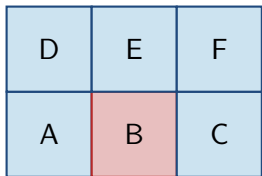


Data: Cressie & Read (1985); spData package

Who Is a Neighbor?

We need to define “nearby” formally

Queen contiguity: two counties are neighbors if they share *any* boundary point (like a queen’s move in chess).



B's neighbors: A, C, E (queen)

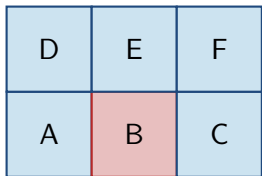
Other options:

- **Rook:** shared edge only (not corners)
- **k -nearest neighbors:** closest k centroids
- **Distance band:** all within d km

Who Is a Neighbor?

We need to define “nearby” formally

Queen contiguity: two counties are neighbors if they share *any* boundary point (like a queen’s move in chess).



B's neighbors: A, C, E (queen)

Other options:

- **Rook:** shared edge only (not corners)
- **k -nearest neighbors:** closest k centroids
- **Distance band:** all within d km

The choice matters — it's a modeling decision!

Building the Neighbor Structure in R

```
# Define neighbors (queen contiguity)
nc_nb <- poly2nb(nc, queen = TRUE)

# Convert to weights (each row sums to 1)
nc_lw <- nb2listw(nc_nb, style = "W")

# The spatial lag: each county's neighbors' average
nc$sids_lag <- lag.listw(nc_lw, nc$sids_rate74)
```

Spatial lag, in plain English

For each county, compute the **average SIDS rate among its neighbors**. If autocorrelation is present, a county's own rate and its neighbors' average should be correlated.

The Moran Scatterplot

```
ggplot(nc, aes(x = sides_rate74, y = sides_lag)) +  
  geom_point(color = "steelblue") +  
  geom_smooth(method = "lm", color = "tomato") +  
  labs(x = "County SIDS rate",  
       y = "Neighbors' average rate")
```

Reading the plot

- **Positive slope:** nearby counties look similar → clustering
- **Flat:** no spatial pattern
- **Negative slope:** checkerboard pattern

The slope of that red line is **Moran's I** , the most common measure of spatial autocorrelation.

Range: roughly -1 to $+1$
(like a correlation coefficient, but for space)

Testing: Is the Clustering Real?

```
# Analytical test  
moran.test(nc$sids_rate74, nc_lw)
```

$$I = \frac{n}{\sum_i \sum_j w_{ij}} \cdot \frac{\sum_i \sum_j w_{ij} (x_i - \bar{x})(x_j - \bar{x})}{\sum_i (x_i - \bar{x})^2}$$

- $I \approx 0$: no spatial autocorrelation (random)
- $I > 0$: positive autocorrelation (clusters of similar values)
- $I < 0$: negative autocorrelation (checkerboard pattern)

Safer: Permutation Test

Shuffle the map a large number of times

```
set.seed(42)
moran.mc(nc$sids_rate74, nc_lw, nsim = 999)
```

How the permutation test works

1. Randomly reassign the 100 SIDS rates to random counties
2. Compute Moran's I for that scrambled map
3. Repeat 999 times to build a “what if spatial pattern were random” distribution
4. Ask: is our observed I unusual compared to those 999 values?

Where Are the Clusters? — LISA

Global Moran's I : one number for the whole map.

LISA (Local Indicators of Spatial Association): tells us *where* clusters are.

-  **High–High:** hot spot
-  **Low–Low:** cold spot
-  **High–Low:** high outlier
-  **Low–High:** low outlier
-  Not significant

In R

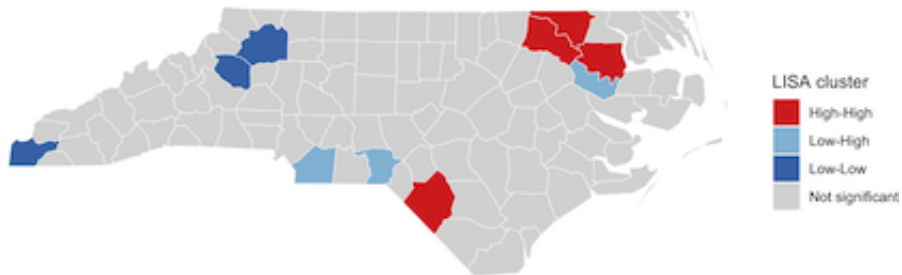
```
lisa <- localmoran(  
  nc$sids_rate74, nc_lw)
```

Discuss

Where do hot spots appear on the NC map? Do they correspond to any geographic or demographic pattern you might expect?

North Carolina LISA Map

Local Moran's I — SIDS Rate Clusters
 $p < 0.05$; NC counties 1974–78



Part 2: Point Patterns — Fiji Earthquakes

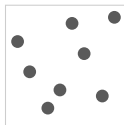
The question

Are the events **clustered**, **random**, or **regular**?

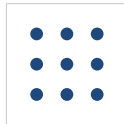
Clustered



Random (CSR*)



Regular



Dataset: quakes (base R) — 1,000 earthquakes near Fiji, magnitude ≥ 4

Visualizing Point Data: Kernel Density

```
# Raw points  
ggplot(quakes,  
  aes(x = long, y = lat)) +  
  geom_point(alpha = 0.3)
```

Kernel density estimation

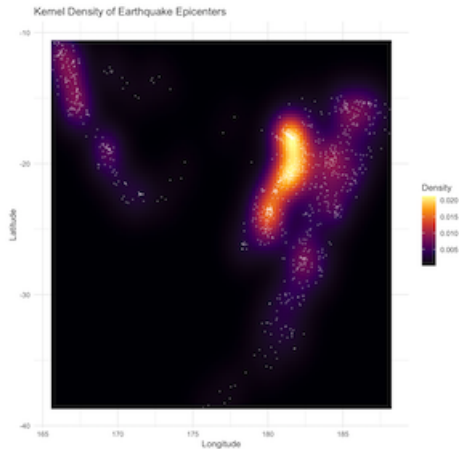
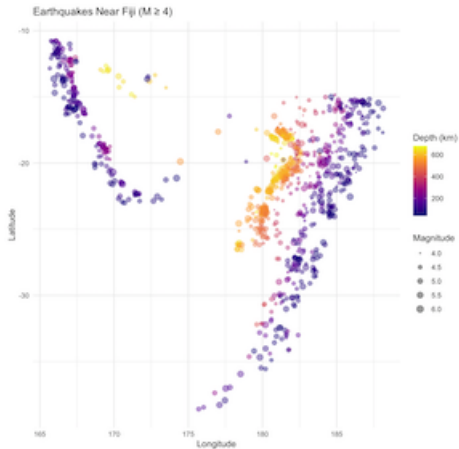
Place a smooth “bump” over each point.
Add them all up. The result is a
continuous surface showing **where**
events are most concentrated.

```
# Density surface  
ggplot(quakes,  
  aes(x = long, y = lat)  
  ) +  
  stat_density_2d(  
    aes(fill=after_stat(  
      density))),  
  geom="raster",  
  contour=FALSE) +  
  scale_fill_viridis_c(  
    option="inferno")
```


Cluster Patterns

Discuss

What pattern do you see? Why might earthquakes cluster near Fiji?



Part 3: Geostatistics — Meuse River Soils

Setup

- 155 soil samples from the Meuse river floodplain (Netherlands)
- Measuring zinc concentration (ppm) at each point
- **Goal:** predict zinc everywhere, not just at sample locations

Why not just take the average?

Zinc concentration isn't uniform — it's higher near the river channel. We want a *spatial* prediction.

Load the data

```
data("meuse", package="sp")  
meuse_sf$log_zinc <-  
  log(meuse_sf$zinc)
```

Log-transform: zinc values are right-skewed

The Variogram: Does Similarity Decay with Distance?

Key idea: Nearby soil samples should have similar zinc levels. Far-apart samples should be less similar.

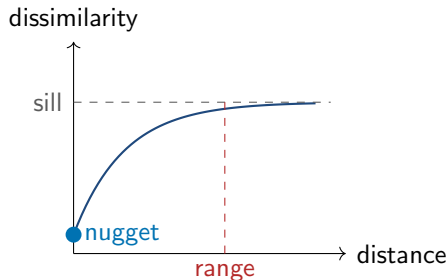
The **variogram** plots:

*average squared difference between pairs
of measurements*

vs.

distance between them

If the curve rises then flattens, spatial correlation exists.



Nugget: residual variance at zero distance

Range: correlation disappears beyond here

Sill: total variance

Computing the Variogram in R

$$\gamma(h) = \frac{1}{2|N(h)|} \sum_{N(h)} [Z(s_i) - Z(s_j)]^2$$

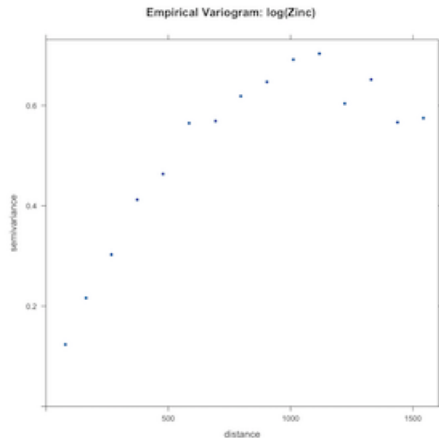
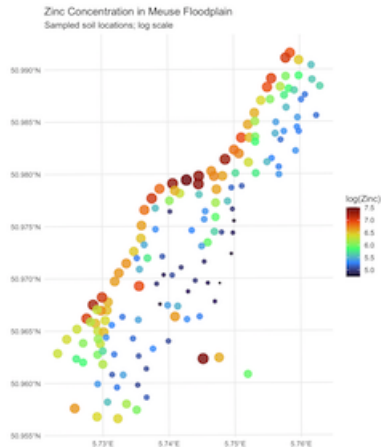
where $N(h)$ is the set of pairs separated by distance h .

```
# Step 1: compute empirical variogram
v_emp <- variogram(log_zinc ~ 1, data = meuse_sp)
plot(v_emp)
# Step 2: fit a model (spherical shape)
v_fit <- fit.variogram(v_emp,
                      model = vgm(psill = 0.6,
                                   model = "Sph",
                                   range = 900,
                                   nugget = 0.05))
plot(v_emp, v_fit)
```

Empirical Variogram Interpretation

Discuss

Does your variogram level off? At roughly what distance? Is the nugget large or small relative to the sill?



Kriging: Spatial Interpolation

What is kriging?

Predict zinc at any unsampled location by taking a **weighted average of nearby observations**. Closer samples get more weight. The variogram tells us exactly how to assign the weights.

```
krig_out <- krige(  
  log_zinc ~ 1, # formula  
  meuse_sp,     # sample data  
  meuse.grid,   # predict here  
  model = v_fit)# variogram
```

Kriging gives you two maps:

- **Prediction:** best estimate of zinc everywhere
- **Variance:** uncertainty of estimates, lowest near sample points

Two Kinds of Kriging

Ordinary Kriging

$\log_zinc \sim 1$

Assumes the mean zinc concentration is **constant** across the whole study area.

All spatial pattern is treated as random variation around that one global mean.

Use when: no obvious directional gradient in your map of raw data

Universal Kriging

$\log_zinc \sim dist$

Allows the mean to **vary** as a function of a covariate (here: distance to river).

First removes the trend, then models spatial structure in what remains.

Use when: a clear gradient is visible, like zinc decreasing away from the river

Why Does the Choice Matter?

What goes wrong with ordinary kriging when there is a trend?

- The variogram absorbs the trend into the sill
- Sill ends up **inflated** above the true sample variance
- Spatial correlation structure is mis-estimated
- Predictions away from data can be biased

In our Meuse example: sill was 0.64 under ~ 1 but only 0.29 under $\sim \text{dist}$

What universal kriging buys you:

- Cleaner variogram fit closer to sample variance
- Shorter, more realistic range estimate
- Predictions that respect the physical gradient
- Lower prediction variance near the trend

In our example: range shrank from 897m to 729m once the river-distance trend was removed

How to Choose: A Simple Workflow

1. **Map your raw data first.**

If you see a clear directional gradient, a trend is present.

2. **Check: does the variogram sill match $\text{var}(y)$?**

A sill much larger than the sample variance suggests a trend is inflating it.

3. **If trend present:** fit $y \sim \text{covariate}$ and recompute the variogram on the residuals.

4. **Compare the two variograms.**

The universal kriging variogram should have a lower sill and fit closer to the sample variance.

5. **Use the matching formula in `krige()`.**

Whatever formula you used in `variogram()`, use the same one in `krige()`.

Practical Limitation of Universal Kriging

Discuss

Universal kriging requires a covariate measured at every prediction location, not just at sample points.

- *What does that mean practically?*
- *What do you need to have in hand before you can run `krige()`?*

What We Covered

Data type	Key idea	Main tool
Areal (NC counties)	Spatial autocorrelation	Moran's I , LISA
Point process (earthquakes)	Clustering vs. random	Kernel density
Geostatistical (Meuse soils)	Predict at new locations	Kriging

The thread running through all three

Nearby things tend to be similar. Each method uses that fact differently:

- Moran's I — *measures* how similar neighbors are
- KDE — *visualizes* where events concentrate
- Kriging — *exploits* similarity to predict in new places

Exercises

Exercise 1

Repeat the Moran's I test for the 1979–84 SIDS period (`sid_rate79`). Does clustering increase or decrease over time?

Exercise 2

Switch to $k = 5$ nearest-neighbor weights instead of queen contiguity. How much does Moran's I change?

```
knn2nb(knearneigh(coords, k = 5))
```

Challenge

Install `spatstat`. Convert quakes to a ppp object and compute the L -function. Is there evidence of clustering at short distances?

Discussion Questions

1. We found that high SIDS counties tend to cluster together. Does that *prove* something spatial is causing it? What else could explain the pattern?
2. The choice of neighbor definition (queen, k -NN, distance band) is a *modeling decision*. What could go wrong if you choose poorly?
3. Kriging is most uncertain far from sample points. If you could add 10 more soil samples to the Meuse study, where would you place them?
4. All three methods assume spatial pattern is *stationary*, which means that the pattern is the same everywhere. When might that fail?

Lovelace, Nowosad & Muenchow — *Geocomputation with R*

Bivand, Pebesma & Gómez-Rubio — *Applied Spatial Data Analysis with R*

Pebesma & Bivand — *Spatial Data Science* (r-spatial.org/book)